

Day-2 Hand Landmark Collection Code Explanation

Date: 16/01/2026

Below is a detailed line-by-line explanation of the Day 2 code used to collect hand landmark data for multiple gestures and save it to CSV files.

Code Line / Section:

```
import cv2
```

Explanation:

Imports OpenCV for capturing video frames from the webcam and displaying the live feed.

```
import mediapipe as mp
```

Explanation:

Imports MediaPipe for hand landmark detection.

```
import csv
```

```
import os
```

Explanation:

csv is used to write landmark data into CSV files. os is used to handle directory and file operations.

```
GESTURE_NAME = 'selam'#(changed it for every csv folders)
```

```
NUM_RECORDINGS = 5
```

```
OUTPUT_DIR = f'data/raw_landmarks/{GESTURE_NAME}'
```

```
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

Explanation:

Defines the gesture label (selam), number of recordings per gesture (5), and output directory to store CSVs. os.makedirs ensures the folder exists.

```
mp_hands = mp.solutions.hands
```

```
hands = mp_hands.Hands(min_detection_confidence=0.7, min_tracking_confidence=0.5)
```

Explanation:

Initializes MediaPipe Hands with confidence thresholds for reliable detection and tracking.

```
cap = cv2.VideoCapture(0)
```

Explanation:

Opens the default webcam for capturing live video frames.

```
print("Press 's' to start recording...")
```

Explanation:

Instructs the user to press 's' to begin a recording session.

```
recording_count = 0
```

```
while recording_count < NUM_RECORDINGS:
```

```
    ret, frame = cap.read()
```

Explanation:

Loops until the desired number of recordings are captured. Reads each frame from the webcam.

```
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
results = hands.process(frame_rgb)
```

Explanation:

Converts the frame to RGB and passes it to MediaPipe Hands to detect landmarks.

```
if results.multi_hand_landmarks:
```

```
    for hand_landmarks in results.multi_hand_landmarks:
```

```
        landmark_list = []
```

```
        for lm in hand_landmarks.landmark:
```

```
            landmark_list.extend([lm.x, lm.y, lm.z])
```

Explanation:

If hands are detected, loops through each hand. Extracts x, y, z coordinates of all 21 landmarks and flattens them into a list.

```
csv_path = os.path.join(OUTPUT_DIR, f'{GESTURE_NAME}_{recording_count}.csv')
```

```
with open(csv_path, 'w', newline='') as f:
```

```
    writer = csv.writer(f)
```

```
    writer.writerow(['x'+str(i) for i in range(len(landmark_list))])
```

```
writer.writerow(landmark_list)
```

Explanation:

Creates a CSV file for each recording. Writes column headers and the flattened landmark data.

```
recording_count += 1
```

```
print(f'Recording {recording_count} done!')
```

Explanation:

Updates the recording counter and informs the user that the current recording is complete.

```
cv2.imshow('Recording', frame)
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

Explanation:

Displays the live frame during recording. Pressing 'q' allows the user to exit early.

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Explanation:

Releases the webcam and closes all OpenCV windows after recording is complete.

This line-by-line explanation allows you to understand exactly how the Day 2 gesture recording and landmark collection code works, including data storage and MediaPipe processing.